

Introduction to Algorithms

Levon Prazyan

Course Overview

- Introduction
- Sorting and Searching
- Arrays and Lists
- Stacks and Queues, Double Ended Queue
- Heaps
- **Trees**
- Hash Tables
- Dynamic Programming
- Greedy Algorithms
- Graph Algorithms
- Network Flow
- NP-Completeness

Binary Tree

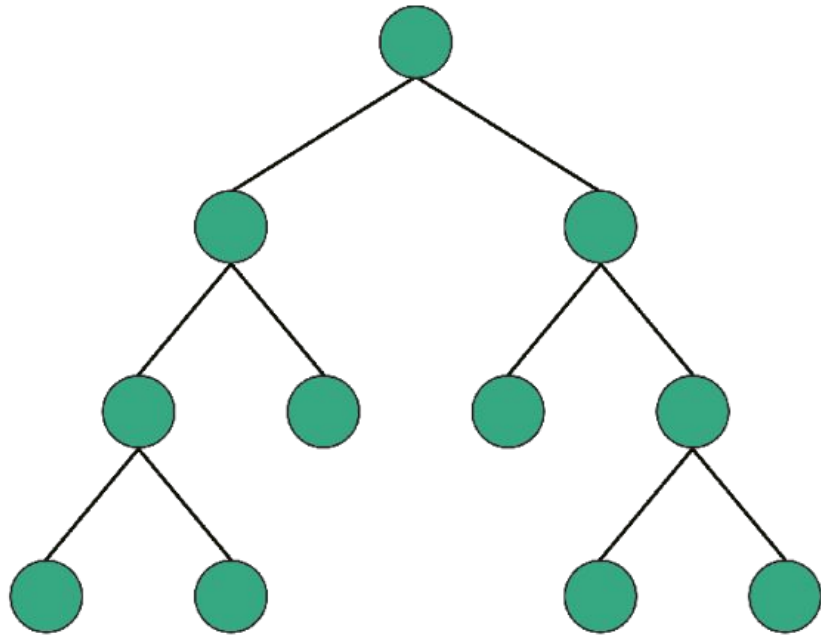
A **binary tree** is a tree data structure in which each node has at most two children, typically referred to as the **left child** and the **right child**.

Key Characteristics:

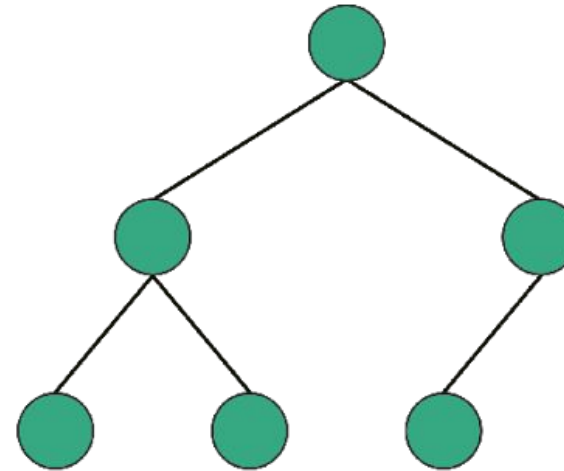
1. Root: The topmost node of the binary tree.
2. Node: Each element in the tree that holds data.
 - Leaf Nodes: Nodes that do not have any children.
 - Internal Nodes: Nodes that have at least one child.
3. Left Child: The left node that is connected to a parent node.
4. Right Child: The right node that is connected to a parent node.

Types of Binary Trees (1)

Full Binary Tree.

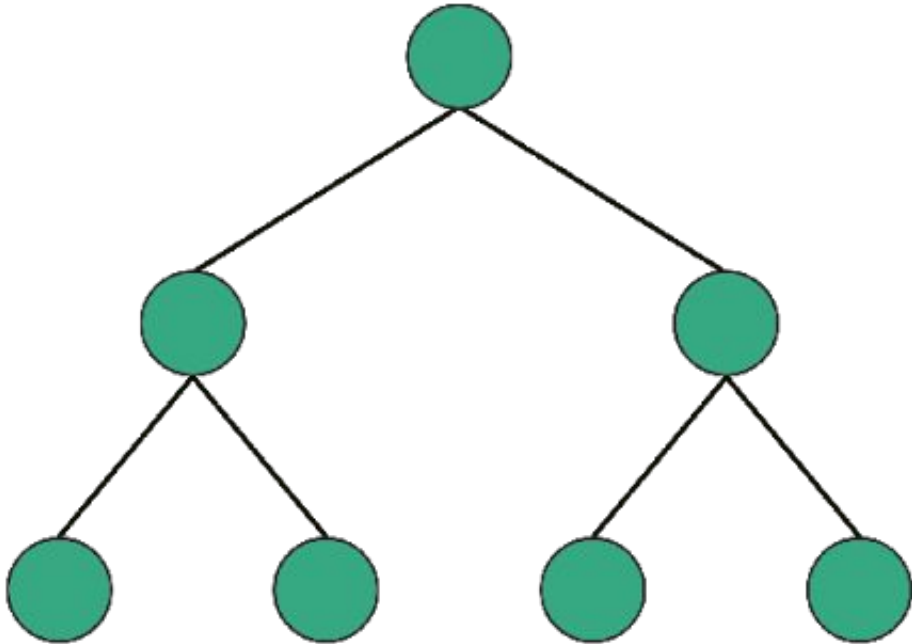


Complete Binary Tree.

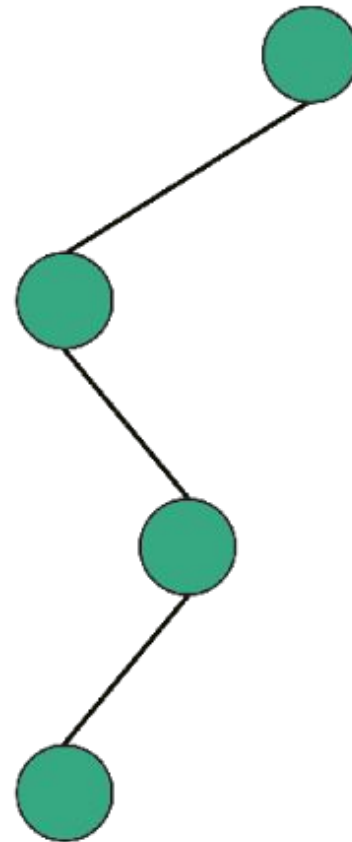


Types of Binary Trees (2)

Perfect Binary Tree.

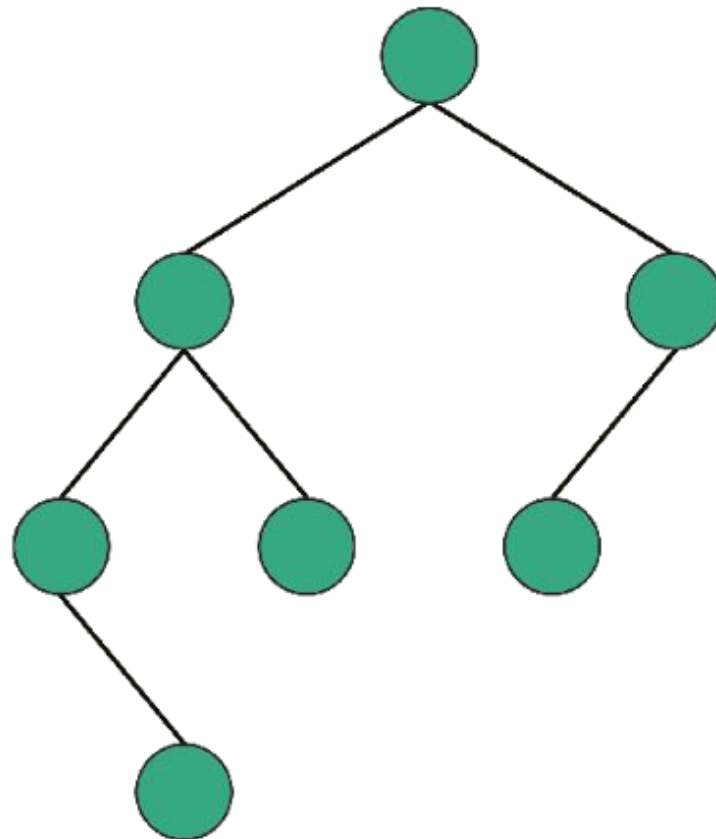


Degenerate Tree.



Types of Binary Trees (3)

Balanced Binary Tree.

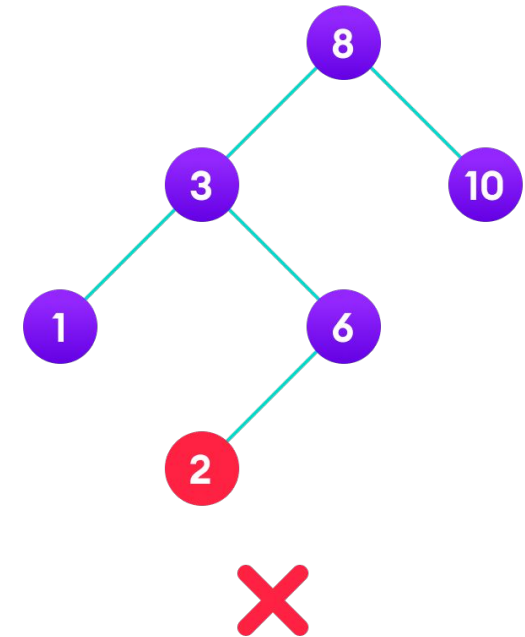
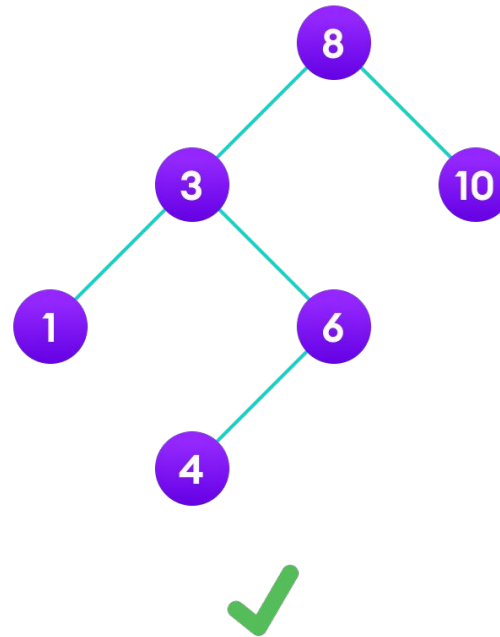


Binary Tree Traversals

- **Pre-order:** Visit the root, traverse the left subtree, then the right subtree.
- **In-order:** Traverse the left subtree, visit the root, then the right subtree.
- **Post-order:** Traverse the left subtree, the right subtree, then visit the root.
- **Level-order:** Traverse the tree level by level (also known as breadth-first traversal).

Binary Search Tree (1)

Binary Search Trees (BST) is a binary tree where for every node, all the elements in the left subtree are less than the node, and all the elements in the right subtree are greater.



Binary Search Tree: Inorder Tree Walk (1)

The binary-search-tree property allows us to print out all the keys in a binary search tree in sorted order by a simple recursive algorithm.

INORDER-TREE-WALK(x)

```
1  if  $x \neq \text{NIL}$ 
2      INORDER-TREE-WALK( $x.\text{left}$ )
3      print  $x.\text{key}$ 
4      INORDER-TREE-WALK( $x.\text{right}$ )
```

Binary Search Tree: Inorder Tree Walk (2)

Theorem: If x is the root of an n -node subtree, then the call *INORDER – TREE – WALK*(x) takes $\theta(n)$ time.

Proof: Let $T(n)$ denote the time taken by *INORDER – TREE – WALK* when it is called on the root of an n -node subtree.

It visits all n nodes, so $T(n) = \Omega(n)$. Need to show that $T(n) = O(n)$.

$T(n) = T(k) + T(n - k - 1) + d$, k is number of nodes for x .*left* tree.

Using substitution method, suppose $T(i) \leq (c + d)i + c$; $0 < i < n$

$$T(n) \leq (c + d)k + c + (c + d)(n - k - 1) + c + d$$

$$T(n) \leq (c + d)n + c; \Rightarrow T(n) = O(n)$$

$$\mathbf{T(n) = \theta(n)}$$

Binary Search Tree: Exercises 1.

1. For the set of $\{2, 4, 5, 7, 8, 9, 10, 15, 17\}$ draw BST of height 3, 4, 5.
2. Give recursive algorithms that perform preorder and postorder tree walks in $\theta(n)$ time on a tree of n nodes.
3. Give an Ω notation for an algorithm that constructs a BST from an array.
4. What is the difference between the BST property and the min-heap property? Can the min-heap property be used to print out the keys of an n -node tree in sorted order in $O(n)$ time? Show how, or explain why not.

Binary Search Tree: Preorder walk

1. Prorder-Tree-Walk(x)
2. if x != null
3. print x.key
4. Preorder-Tree-Walk(x.left)
5. Preorder-Tree-Walk(x.right)

Binary Search Tree: Postorder walk

1. Prorder-Tree-Walk(x)
2. if x != null
3. Postorder-Tree-Walk(x.left)
4. Postorder-Tree-Walk(x.right)
5. print x.key

Querying a Binary Search Tree

Common operations that Binary Search Tree supports

- Tree-Search
- Minimum
- Maximum
- Successor
- Predecessor

All operations require $O(h)$ time, where h is a height of the tree.

Binary Search Tree: Searching (1)

TREE-SEARCH(x, k)

1 **if** $x == \text{NIL}$ or $k == x.\text{key}$

2 **return** x

3 **if** $k < x.\text{key}$

4 **return** TREE-SEARCH($x.\text{left}, k$)

5 **else return** TREE-SEARCH($x.\text{right}, k$)

Binary Search Tree: Searching (2)

ITERATIVE-TREE-SEARCH(x, k)

```
1  while  $x \neq \text{NIL}$  and  $k \neq x.\text{key}$   
2      if  $k < x.\text{key}$   
3           $x = x.\text{left}$   
4      else  $x = x.\text{right}$   
5  return  $x$ 
```


Binary Search Tree: Min and Max

TREE-MINIMUM(x)

```
1  while  $x.left \neq \text{NIL}$   
2       $x = x.left$   
3  return  $x$ 
```

TREE-MAXIMUM(x)

```
1  while  $x.right \neq \text{NIL}$   
2       $x = x.right$   
3  return  $x$ 
```

Binary Search Tree: Successor

TREE-SUCCESSOR(x)

```
1  if  $x.right \neq \text{NIL}$ 
2      return TREE-MINIMUM( $x.right$ )
3   $y = x.p$ 
4  while  $y \neq \text{NIL}$  and  $x == y.right$ 
5       $x = y$ 
6       $y = y.p$ 
7  return  $y$ 
```

Binary Search Tree: Exercises 2 (1)

1. Suppose that we have numbers between 1 and 1000 in a binary search tree, and we want to search for the number 363. Which of the following sequences could not be the sequence of nodes examined?
 - a) 2, 252, 401, 398, 330, 344, 397, 363.
 - b) 924, 220, 911, 244, 898, 258, 362, 363.
 - c) 925, 202, 911, 240, 912, 245, 363.
 - d) 2, 399, 387, 219, 266, 382, 381, 278, 363.
 - e) 935, 278, 347, 621, 299, 392, 358, 363

Binary Search Tree: Exercises 2 (2)

1. Write the *TREE – PREDECESSOR* procedure.
2. Show that if a node in a binary search tree has two children, then its successor has no left child and its predecessor has no right child.
3. An alternative method of performing an **inorder tree walk** of an n -node binary search tree finds the minimum element in the tree by calling *TREE – MINIMUM* and then making $n - 1$ calls to *TREE – SUCCESSOR*. Write and give Θ .
4. Prove that no matter what node we start at in a height- h binary search tree, k successive calls to *TREE – SUCCESSOR* take $O(h + k)$ time.

Binary Search Tree: Predecessor

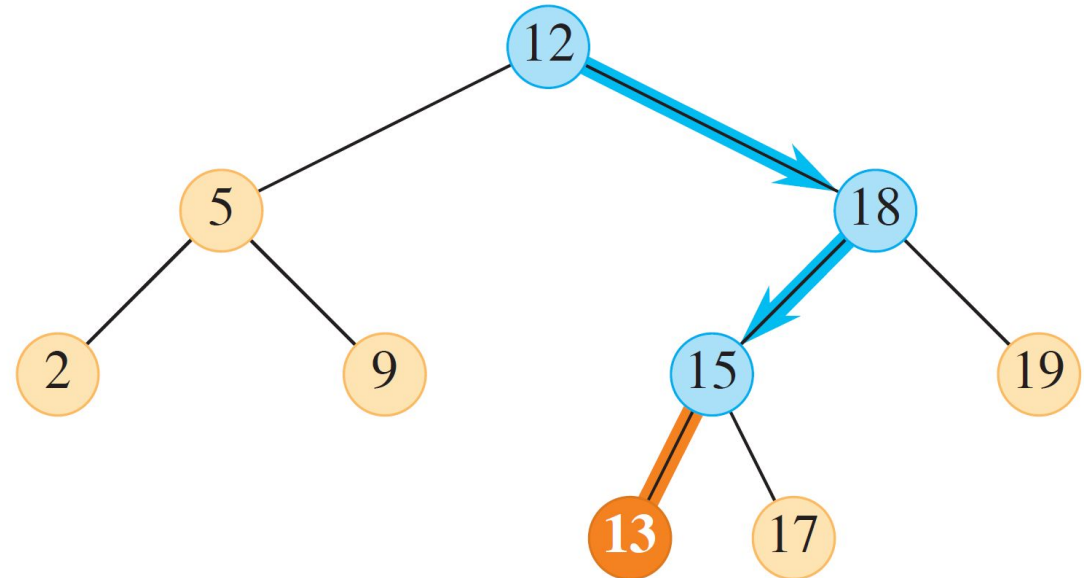
Tree_Predecessor(x)

1. **if** x.left != null
2. **return** Tree_Maximum(x.left)
3. y = x.p
4. **while** y != null **and** x == y.left
5. x = y
6. y = x.p
7. **return** y

Binary Search Tree: Insertion

TREE-INSERT(T, z)

```
1   $y = \text{NIL}$ 
2   $x = T.\text{root}$ 
3  while  $x \neq \text{NIL}$ 
4       $y = x$ 
5      if  $z.\text{key} < x.\text{key}$ 
6           $x = x.\text{left}$ 
7      else  $x = x.\text{right}$ 
8   $z.p = y$ 
9  if  $y == \text{NIL}$ 
10      $T.\text{root} = z$ 
11  elseif  $z.\text{key} < y.\text{key}$ 
12      $y.\text{left} = z$ 
13  else  $y.\text{right} = z$ 
```

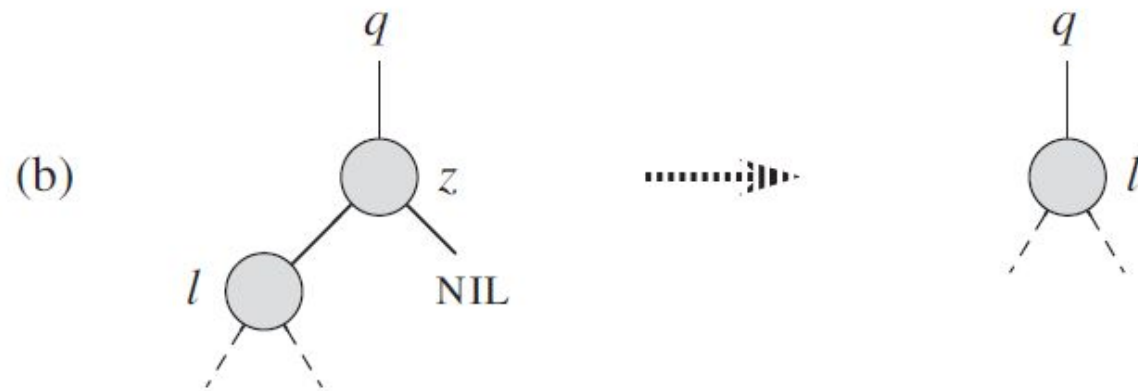
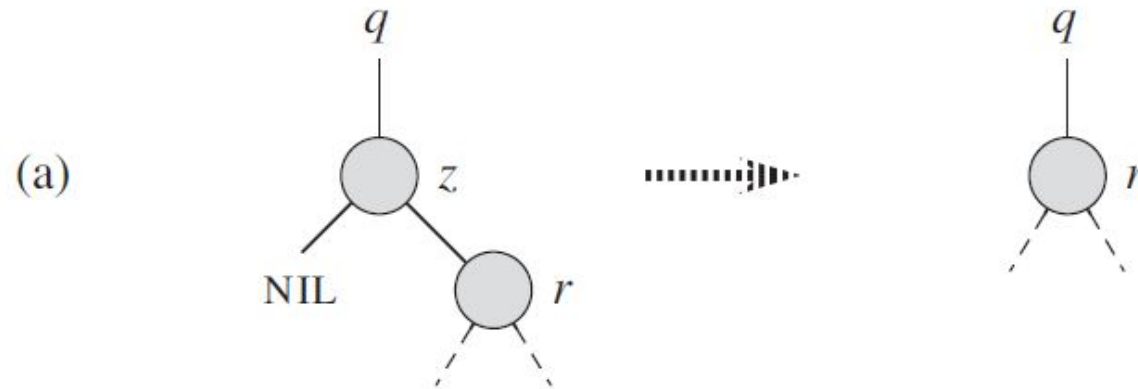


Binary Search Tree: Deletion (1)

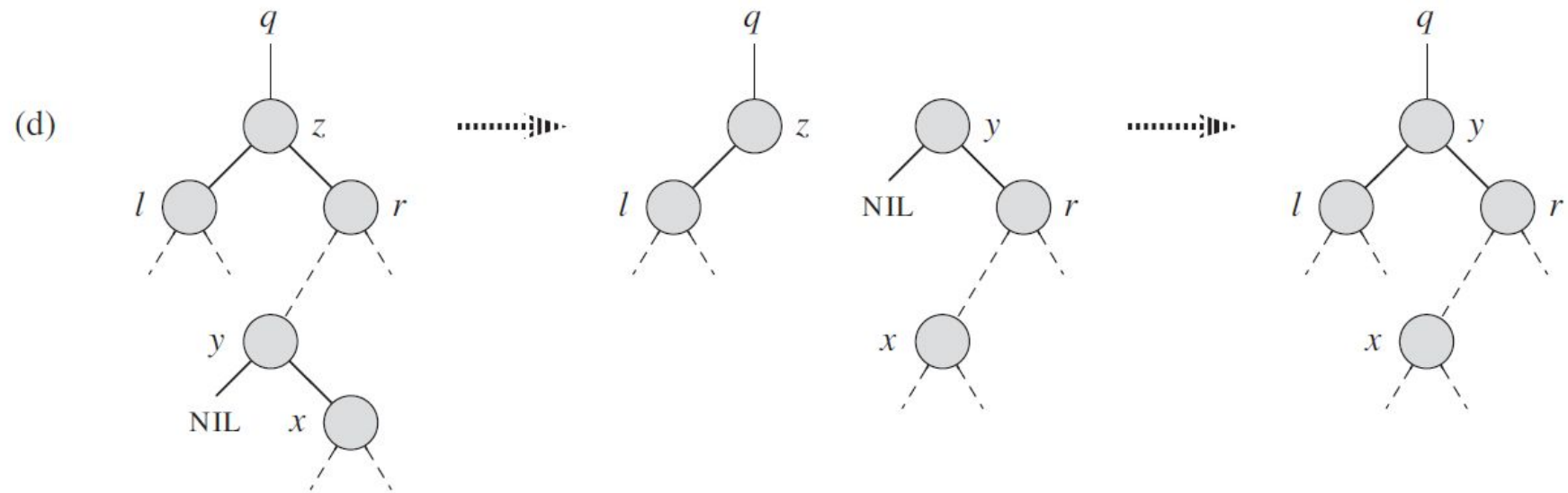
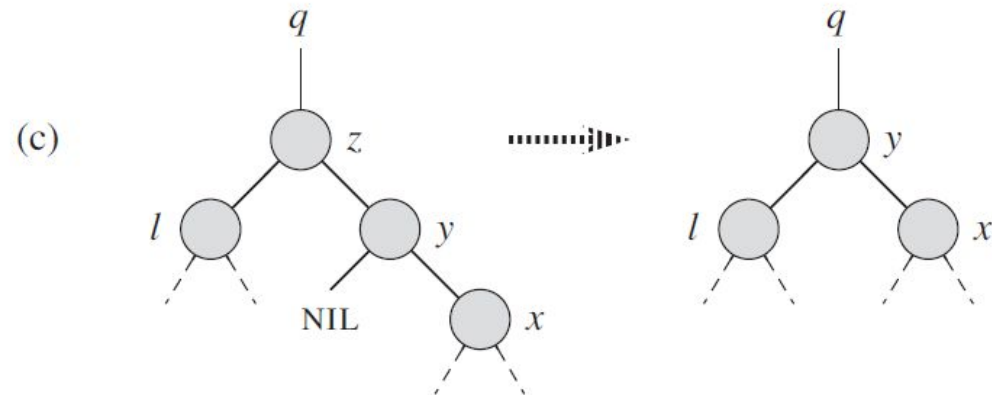
There are 3 basic cases when deleting a node z from tree T

1. If z has no children, then we simply remove it by modifying its parent to replace with *NIL* as its child.
2. If z has just one child, then we elevate that child to take z 's position in the tree by modifying z 's parent to replace z by z 's child.
3. If z has two children, then we find z 's successor y —which must be in z 's right subtree—and have y take z 's position in the tree. The rest of z 's original right subtree becomes y 's new right subtree, and z 's left subtree becomes y 's new left subtree. This case is the tricky one because, as we shall see, it matters whether y is z 's right child.

Binary Search Tree: Deletion (2)



Binary Search Tree: Deletion (3)



Binary Search Tree: Deletion (4)

TRANSPLANT(T, u, v)

```
1  if  $u.p == \text{NIL}$ 
2       $T.\text{root} = v$ 
3  elseif  $u == u.p.\text{left}$ 
4       $u.p.\text{left} = v$ 
5  else  $u.p.\text{right} = v$ 
6  if  $v \neq \text{NIL}$ 
7       $v.p = u.p$ 
```

TREE-DELETE(T, z)

```
1  if  $z.\text{left} == \text{NIL}$ 
2      TRANSPLANT( $T, z, z.\text{right}$ )
3  elseif  $z.\text{right} == \text{NIL}$ 
4      TRANSPLANT( $T, z, z.\text{left}$ )
5  else  $y = \text{TREE-MINIMUM}(z.\text{right})$ 
6      if  $y.p \neq z$ 
7          TRANSPLANT( $T, y, y.\text{right}$ )
8           $y.\text{right} = z.\text{right}$ 
9           $y.\text{right}.p = y$ 
10     TRANSPLANT( $T, z, y$ )
11      $y.\text{left} = z.\text{left}$ 
12      $y.\text{left}.p = y$ 
```

Binary Search Tree: Exercises 3 (2)

1. Is the operation of deletion “commutative” in the sense that deleting x and then y from a binary search tree leaves the same tree as deleting y and then x ?
Argue why it is or give a counterexample.
2. Validate Binary Search Tree: Write a function to check if a binary tree is a valid BST.
3. Find Lowest Common Ancestor: Given a BST and two node values, return the lowest common ancestor.
4. Find K^{th} Smallest Element: Write a function to return the k -th smallest element in a BST.



Thank You